

CPU Architecture Verification on Kali Linux

Checking 64-bit support before choosing the amd64 Kali image | Lab Report (KLCP Lab 1)

1. Overview

This report documents a short terminal exercise that confirms the processor architecture of a Kali Linux system. Choosing the correct ISO (for example `kali-linux-2026.2-live-amd64.iso`) depends on knowing whether the machine is 64-bit. Three methods are used: the kernel-reported machine name (`uname -m`), the raw CPU flags (`/proc/cpuinfo`), and a regular-expression test for the `lm` (long mode) flag.

2. Learning Objectives

- Identify the system architecture from the command line.
- Read CPU information from the `/proc` virtual filesystem.
- Interpret CPU feature flags, especially `lm` and `hypervisor`.
- Use `grep` options (`-m`, `-q`, `-P`) and regular expressions.
- Use shell operators `&&` and `||` for conditional output.

3. Environment

- Kali Linux terminal; prompt `kali@kali`, working directory `~` (home).
- `x86_64` architecture reported by `uname -m`.
- The CPU flags include `hypervisor`, which indicates the system is running under a hypervisor (virtual machine).

4. Technical Concepts

x86_64 and long mode

`x86_64` (also called `amd64`) is the 64-bit extension of the `x86` instruction set. The CPU flag `lm` stands for **long mode**, the CPU mode required to run 64-bit code. A 32-bit-only processor lacks this flag.

The `/proc` filesystem

`/proc` is a virtual filesystem created by the kernel. Files such as `/proc/cpuinfo` are not stored on disk; they are generated on read and describe the processor (model, cores, and the `flags` line listing supported features).

Exit status and conditional operators

Every command returns an exit status: 0 for success, non-zero for failure. `grep -q` returns 0 if it found a match. `cmd1 && cmd2 || cmd3` runs `cmd2` when `cmd1` succeeds, otherwise `cmd3`.

5. Practical Procedure

Step 1 — Read the machine architecture

Objective: get the architecture name reported by the kernel.

```
uname -m
```

Component	Meaning
<code>uname</code>	Prints system information from the kernel.
<code>-m</code>	Machine hardware name (the CPU architecture).

Output: `x86_64` — a 64-bit `x86` system, matching the `amd64` image family.

Step 2 — Inspect the CPU flags

Objective: view the processor feature flags directly.

```
grep -m1 "flags" /proc/cpuinfo
```

Component	Meaning
grep	Searches text for lines matching a pattern.
-m1	Stop after the first 1 matching line. Without it, grep prints one flags line per CPU core.
"flags"	The search pattern; the matched word is highlighted red in the terminal.
/proc/cpuinfo	Kernel-generated CPU description file.

Output analysis

- `lm` — long mode, the 64-bit capability. It appears in the sequence `syscall nx rdtscp lm constant_tsc`.
- `pae` (Physical Address Extension), `nx` (no-execute bit) and `syscall` (fast system-call instruction) are also present.
- `sse`, `sse2`, `sse4_1`, `sse4_2`, `avx`, `avx2` and `aes` list SIMD and encryption instruction support.
- `hypervisor` — the system is running as a guest under a hypervisor.
- Part of the line is cut off at the right edge of the terminal window; the `lm` flag is fully visible.

Step 3 — Test for the lm flag

Objective: turn the flag check into a clear 64-bit / 32-bit answer.

```
grep -qP '^flags\s*:.*\blm\b' /proc/cpuinfo && echo 64-bit || echo 32-bit
```

Component	Meaning
-q	Quiet: print nothing, only set the exit status.
-P	Use Perl-compatible regular expressions (needed for <code>\s</code> and <code>\b</code>).
<code>^flags</code>	Line must start with the word <code>flags</code> .
<code>\s*</code>	Zero or more whitespace characters.
<code>:</code>	The literal colon after the label.
<code>.*</code>	Any characters in between.
<code>\blm\b</code>	The word <code>lm</code> with word boundaries, so it will not match inside another flag name.
<code>&&</code>	Run the next command only if grep found a match (exit status 0).
<code>echo 64-bit</code>	Prints 64-bit.
<code> </code>	Run the next command only if the previous part failed.
<code>echo 32-bit</code>	Prints 32-bit if no <code>lm</code> flag was found.

Output: 64-bit. The pattern matched, so the CPU reports long mode.

Step 4 — Repeat the checks

Both `uname -m` and the `lm` test were run a second time and returned the same results, `x86_64` and `64-bit`, which confirms the result is consistent.

```

(kali@kali)-[~]
$ uname -m
x86_64

(kali@kali)-[~]
$ grep -m1 "flags" /proc/cpuinfo
flags       : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 h
syscall nx rdtscp lm constant_tsc rep_good noopl xtopology nonstop_tsc cpuid tsc_known_freq pni pclmulqdq ssse3 fma c
16 pcid sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ibrs_enhanced fsgsbs
se bmi1 avx2 bmi2 invpcid rdseed adx clflushopt sha_ni arat md_clear flush_l1d arch_capabilities

(kali@kali)-[~]
$ grep -qP '^flags\s*:.*\blm\b' /proc/cpuinfo && echo 64-bit || echo 32-bit
64-bit

(kali@kali)-[~]
$ uname -m
x86_64

(kali@kali)-[~]
$ grep -qP '^flags\s*:.*\blm\b' /proc/cpuinfo && echo 64-bit || echo 32-bit
64-bit

```

Screenshot 1 — uname -m, CPU flags from /proc/cpuinfo, and the lm test printing 64-bit

6. Practical Workflow

1. `uname -m`: `x86_64`
2. Read the flags line in `/proc/cpuinfo`
3. Find `lm` (long mode) among the flags
4. Run the regex test: 64-bit
5. Repeat to confirm; choose the amd64 image

7. Command Reference

Command	Purpose	Important options	Note
<code>uname -m</code>	Show architecture	<code>-m</code>	<code>x86_64</code> means 64-bit x86
<code>grep -m1 "flags" /proc/cpuinfo</code>	Show first CPU flags line	<code>-m1</code>	One line instead of one per core
<code>grep -qP '^flags\s*:.*\blm\b' ...</code>	Test for long mode	<code>-q</code> , <code>-P</code>	Exit status drives the echo
<code>&& / </code>	Conditional execution	none	Success path / failure path

8. Screenshot Index

Screenshot	Description	Related Step
Screenshot 1	<code>uname -m</code> output, <code>grep</code> of the flags line, <code>lm</code> test printing 64-bit (run twice)	Steps 1–4

9. Security Considerations

- Verifying architecture first prevents downloading and writing an image that will not boot on the machine.
- `/proc/cpuinfo` is read-only information; none of these commands change the system.
- The hypervisor flag is a reminder that results describe the virtual CPU presented to the guest, not necessarily the physical host.

10. Key Takeaways

- Two independent checks (`uname -m` and the `lm` flag) agree on 64-bit.
- Regular expressions with word boundaries avoid false matches on flag names.

- Exit statuses combined with && and | | make compact test commands.

11. Conclusion

The system was confirmed as x86_64 with long-mode support, so the amd64 Kali Live ISO is the appropriate image for this machine. The exercise practiced reading kernel-provided CPU data, interpreting feature flags and writing a conditional shell test. It is a foundational exercise and does not by itself represent professional experience.